



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет автоматики
и вычислительной техники**

Кафедра вычислительной техники
Лабораторная работа № 4
по дисциплине «Технология разработки программного обеспечения»

Java. Разработка оконного приложения

Студент Чумаков Илья

Группа АТМ-25

Преподаватель Токарев Вадим Геннадьевич

Новосибирск, 2025

1. Цель работы

Разработать фабрику классов типов данных (UserFactory), реализующую паттерн «Прототип» в сочетании с элементами фабричного метода, для обеспечения гибкого создания и управления объектами различных пользовательских типов данных (UserType) в оконном приложении.

2. Задание

Необходимо разработать (взять из л.р.3):

2.1. Интерфейс UserType

Тип данных должен предоставлять методы:

- имя типа,
- создание пустого значения,
- клонирование значения,
- чтение/парсинг значения,
- предоставление компаратора для сортировки.

2.2. Фабрику типов данных

Фабрика должна:

1. возвращать список названий типов (для выпадающего списка окна),
2. по имени возвращать объект-прототип соответствующего типа данных.

2.3. Структуру данных

В работе используется хэш-таблица.

Оно должно уметь:

- вставлять элементы;
- получать элементы по индексу;
- удалять элементы по индексу;
- возвращать все элементы для заполнения UI элементов;
- сортироваться с помощью компаратора, полученного от выбранного типа данных.

2.4. Оконный контроллер

- Получает имя выбранного типа;
- Запрашивает у фабрики прототип;
- Использует его для операций: вставка, удаление, парсинг;
- Использует компаратор выбранного типа для сортировки таблицы;
- Отображает элементы сразу после добавления или удаления.

3. Реализация

3.1. Интерфейсы

Интерфейс сравнения:

```
public interface Comparator {  
    int compare(Object o1, Object o2);  
}
```

Интерфейс типа данных:

```
public interface UserType {  
    String typeName();  
  
    Object create();  
  
    Object clone(Object obj);  
  
    Object readValue(InputStreamReader in) throws IOException;  
  
    Object parseValue(String ss);  
  
    Comparator getTypeComparator();  
}
```

3.2. Реализация пользовательских типов

DoubleUserType

Описывает данные типа double.

- Поддерживает парсинг строки в Double через Double.parseDouble(),
- Использует Double.compare() для компаратора.

SparseMatrixUserType

Тип данных — разреженная матрица в координатном формате.

Поддерживает:

- матрицы произвольного размера;
- хранение только ненулевых элементов;
- Ввод строкой вида "n m x1 y1 v1 x2 y2 v2 ...";
- глубокое клонирование всей структуры матрицы;
- сравнение по норме Фробениуса.

3.3. Фабрика типов данных

```
public class UserFactory {
    private final ArrayList<UserType> types = new ArrayList<>();

    public UserFactory() {
        types.add(new DoubleType());
        types.add(new SparseMatrixType());
    }

    public ArrayList<String> getTypeNameList() {
        ArrayList<String> names = new ArrayList<>();
        for (UserType t : types) names.add(t.typeName());
        return names;
    }

    public UserType getBuilderByName(String name) {
        for (UserType t : types)
            if (t.typeName().equals(name))
                return t;
        return null;
    }
}
```

Фабрика автоматически регистрирует все доступные типы при создании. Метод `getTypeNameList()` используется для заполнения выпадающего списка в окне. Метод `getBuilderByName()` возвращает объект-прототип выбранного типа.

3.4. Структура данных – хеш-таблица

Класс `HashTable` обеспечивает:

- хранение элементов произвольного типа через `Object`;
- линейное индексирование элементов независимо от распределения по корзинам
- операции вставки (`addToEnd`, `insert`), удаления (`delete`) и получения (`get`) по логическому индексу;

- обход всех элементов через `forEach` и `getAllObjects()` для отображения в UI;
- создание новой отсортированной таблицы методом `sort()`, использующим компаратор, полученный от текущего `UserType`.

3.5. Оконный контроллер (GUI)

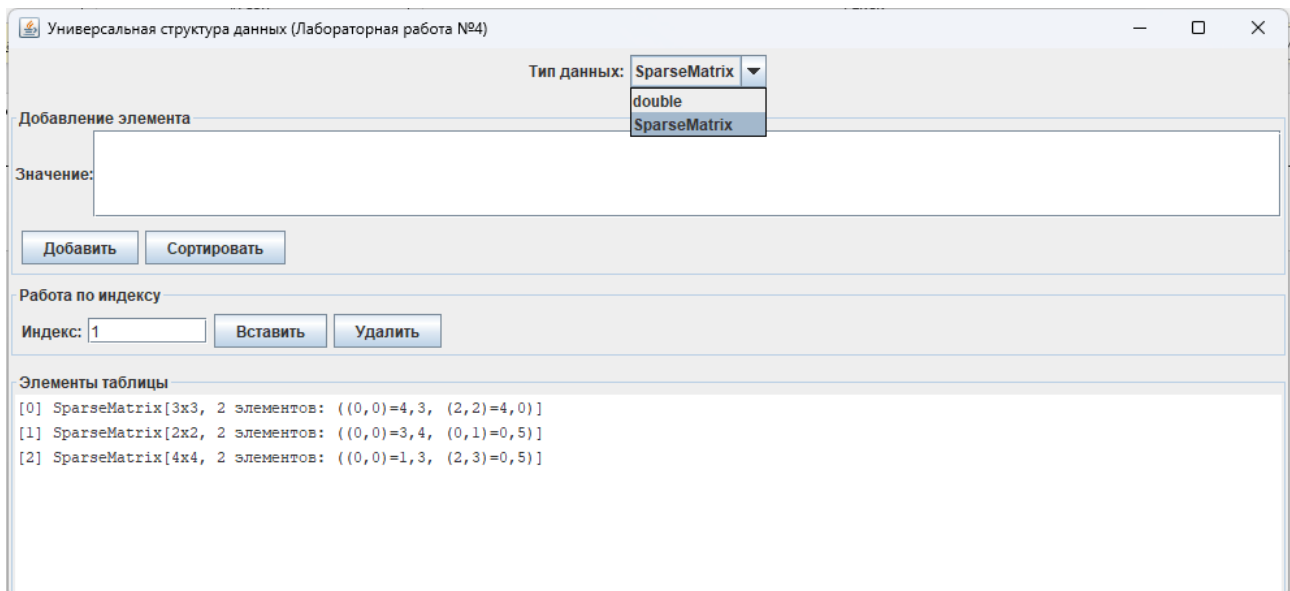
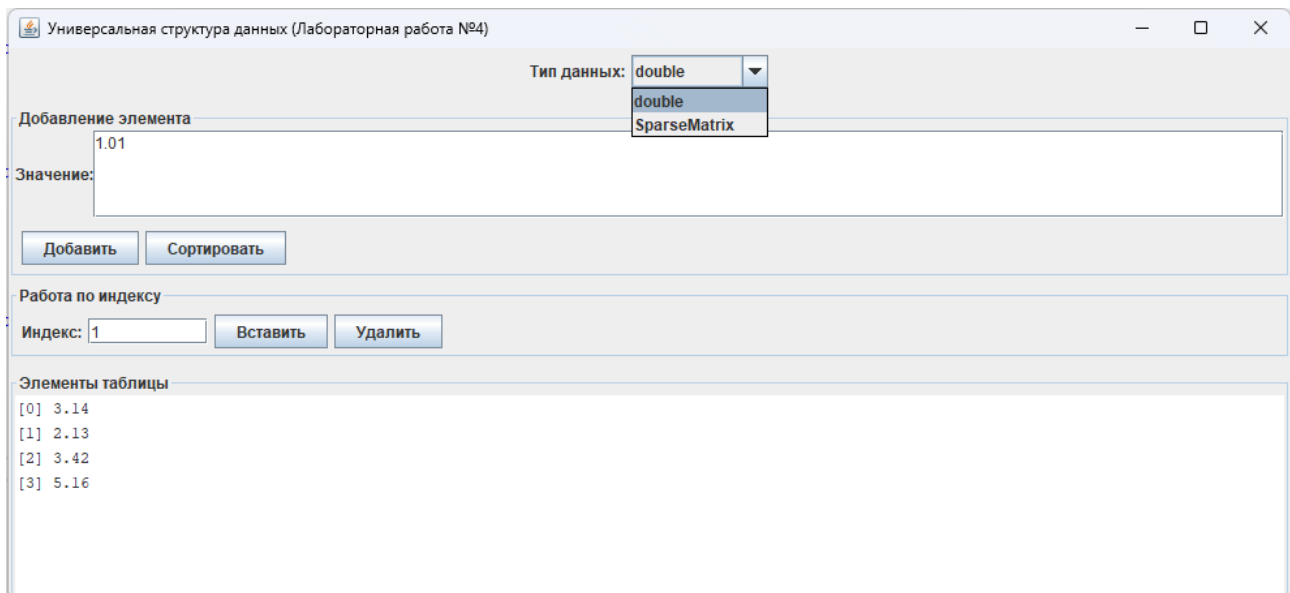
Разработан класс `MainWindow` реализует пользовательский интерфейс:

- В верхней части — выпадающий список с именами типов, полученными из фабрики.
- При выборе типа приложение запрашивает у фабрики соответствующий объект-прототип `UserType` и создаёт новую пустую хеш-таблицу с этим типом.
- Поле ввода позволяет ввести значение в строковом формате, соответствующем текущему типу.
- Кнопка «Добавить» парсит введённую строку и добавляет полученный объект в таблицу.
- Кнопка «Сортировать» создаёт и подменяет текущую таблицу отсортированной версией (используется компаратор типа).
- Панель работы по индексу позволяет вставлять и удалять элементы по произвольному логическому индексу.
- Нижняя часть окна содержит список всех элементов таблицы с указанием их индекса и строкового представления.
- Список автоматически обновляется после каждой операции добавления, вставки, удаления или сортировки.

3.6. Результаты работы программы

При запуске открывается окно с выбором типа данных (`double` или `SparseMatrix`).

- Элементы добавляются через парсинг строки с использованием прототипа выбранного типа.
- После нажатия кнопки «Сортировать» все элементы таблицы упорядочиваются в соответствии с компаратором типа и сразу отображаются в новом порядке.
- Поддерживаются операции вставки и удаления по индексу с обновлением отображаемого списка.
- Фабрика типов позволяет легко расширять приложение новыми типами данных без изменения основного кода.



Запуск программы происходит при запуске файла MainWindow.java.